Guia de apresentação — Equipe 1

Máquina de Turing: multiplicação de números unários. Duração total: 15 minutos.

Este guia explica o que mostrar, o que falar e o significado de cada parte. As falas são sugestões: ensaie com suas palavras e use as explicações para responder ao professor. O roteiro principal vai até o minuto 13; os dois minutos finais ficam para perguntas. As explicações extras e as respostas da parte final são material de estudo, não um texto para ler inteiro durante os 15 minutos.

Antes da apresentação

Abra a pasta do trabalho no editor e deixe disponíveis `simulador.py`, `multiplicacao.txt` e `diagrama_estados.svg`. Abra o SVG no navegador para poder ampliar. Deixe um terminal aberto nessa mesma pasta. Teste `python simulador.py "1*1"` antes de começar, para confirmar que o Python está funcionando.

Ensaie uma vez com cronômetro. Se passar do tempo, reduza a leitura do código e explique as etapas da máquina pelo exemplo. Todos os integrantes precisam entender o conjunto, pois o PDF informa que as perguntas e a nota da apresentação são individuais.

1. Abertura e problema — 0:00 a 1:30

O que mostrar: o exemplo `111*11` $\rightarrow$ `111111`, escrito na tela ou no relatório.

Fala sugerida:

"Nossa equipe ficou com o transdutor de multiplicação. O trabalho recebe dois números em notação unária, separados por um asterisco, e deixa na fita o resultado da multiplicação. Em unário, o valor é a quantidade de símbolos 1. Então, 111 representa três, 11 representa dois e a entrada 111*11 representa três vezes dois. A saída deve ter seis símbolos 1."

"A Máquina de Turing possui uma fita, um cabeçote e um estado atual. O cabeçote lê uma célula por vez. A regra correspondente determina o que escrever, para onde mover o cabeçote e qual será o próximo estado."

O que você precisa entender: `111` não representa cento e onze nem um número binário. Neste trabalho, cada `1` é uma unidade. O asterisco é apenas um símbolo separador; o Python não interpreta essa entrada como uma expressão matemática.

Transição para o próximo assunto: "Primeiro vou mostrar como representamos essa máquina no programa; depois explico como as regras realizam a multiplicação."

2. Arquitetura e fita infinita — 1:30 a 3:30

O que mostrar: em `simulador.py`, procure `fita = dict(enumerate(palavra))`, `fita.get(cabeca, branco)` e `fita.pop(cabeca, None)`, dentro de `simular`. Mostre também que `cabeca` começa em zero e é atualizada pelos movimentos.

Fala sugerida:

"Usamos um dicionário para representar a fita. Cada chave é uma posição inteira e o valor é o símbolo guardado nela. Na entrada 111*11, as posições começam em zero: as posições zero, um e dois contêm 1; a posição três contém o asterisco."

"Não criamos uma lista com um tamanho máximo. Se o cabeçote for para uma posição que ainda não existe no dicionário, a leitura devolve B, que significa branco. As posições podem ser negativas, então também podemos avançar para a esquerda da posição zero."

"Quando escrevemos um branco, removemos a posição do dicionário. Dessa forma, guardamos somente as células não brancas, sem acumular todas as posições vazias que foram visitadas. A fita cresce conforme a necessidade, respeitando a memória disponível no computador."

Exemplo para você entender: `{0: '1', 1: '*', 2: '1'}` representa `1*1`. A posição `-1`, que não está nesse dicionário, é lida como branco. Se a máquina escrever `X` nela, passa a existir a entrada `-1: 'X'`. Não é necessário deslocar todas as outras células.

Ponto importante para a defesa: "infinita" é a propriedade do modelo teórico. A implementação permite crescer nos dois sentidos sem estabelecer um tamanho fixo; isso não significa que a memória física do computador seja infinita.

3. Arquivo de regras e determinismo — 3:30 a 5:00

O que mostrar: abra `multiplicacao.txt`. Aponte as seis definições iniciais e, depois, a regra `q_escolher,1,X,R,q_ir_mult`.

Fala sugerida:

"As regras são lidas deste arquivo de texto. Primeiro declaramos os estados, o alfabeto da entrada, o alfabeto da fita, o branco, o estado inicial e os estados finais. Depois declaramos as transições, no formato exigido pelo enunciado."

"Esta regra significa: se o estado atual for q_escolher e o símbolo lido for 1, escreva X nessa célula, mova uma posição para a direita e entre no estado q_ir_mult. R significa direita, L significa esquerda e S significa ficar na mesma posição."

"O simulador guarda cada regra usando o par estado e símbolo lido como chave. Para esse par pode existir no máximo uma regra. Se houver duas, o arquivo é recusado. É isso que garante que a máquina seja determinística."

"A multiplicação foi modelada nas transições. O motor em Python é genérico: ele não conta os fatores para calcular o produto diretamente, apenas executa as regras do arquivo."

O que você precisa entender: alfabeto de entrada são os símbolos que o usuário pode digitar, `1` e `*`. O alfabeto da fita inclui também marcas usadas durante o trabalho: `X`, `Y`, `#` e `B`. Essas marcas não são entradas válidas do usuário.

4. Lógica da máquina e diagrama — 5:00 a 8:30

O que mostrar: abra `diagrama_estados.svg` e amplie os estados citados. Apresente os grupos abaixo seguindo a ordem. Não é necessário ler todas as setas: explique a função dos grupos e leia uma transição como exemplo.

Fala de introdução:

"A ideia é fazer adições repetidas. Para cada unidade do primeiro fator, copiamos todas as unidades do segundo fator para uma área de resultado na mesma fita. Em três vezes dois, copiamos dois símbolos 1 por rodada, durante três rodadas."

4.1 Validar e preparar a fita

Estados para apontar: `q0`, `q_primeiro`, `q_segundo` e `q_inicio`.

Fala sugerida:

"Primeiro percorremos a entrada e verificamos se os fatores estão separados por um único asterisco. Ao chegar ao branco depois do segundo fator, escrevemos um sustenido. Ele marca o início da área de

resultado. Depois, o cabeçote volta ao começo."

Exemplo para acompanhar: `111*11` passa a ter a forma `111*11#`. Tudo que for acrescentado como resultado ficará à direita de #.

4.2 Escolher uma unidade do primeiro fator

Estados para apontar: `q_escolher` e `q_ir_mult`.

Fala sugerida:

"Escolhemos o próximo 1 do primeiro fator e o trocamos por X. O X indica que essa unidade já foi usada para iniciar uma rodada. Depois atravessamos o restante do primeiro fator e o asterisco para alcançar o segundo fator. Nas próximas rodadas, pulamos os X."

Exemplo: `111*11#` passa a `X11*11#`.

4.3 Copiar o segundo fator

Estados para apontar: `q_copiar`, `q_ir_fim` e `q_voltar_copia`.

Fala sugerida:

"No segundo fator, trocamos um 1 por Y. Essa marca indica que essa unidade já foi copiada na rodada atual. Caminhamos até o primeiro branco depois da área de resultado e escrevemos um 1. Voltamos ao último Y para procurar a próxima unidade que falta copiar."

"Como o segundo fator tem duas unidades, fazemos isso duas vezes. Depois da primeira cópia temos X11*Y1#1. Depois da segunda, temos X11*YY#11."

O que você precisa entender: a máquina usa um único cabeçote, por isso precisa ir e voltar. Ela não lê o segundo fator e escreve no resultado ao mesmo tempo.

4.4 Restaurar e repetir

Estados para apontar: `q_restaurar` e `q_voltar_esquerda`.

Fala sugerida:

"Quando encontramos o sustenido, terminamos a cópia do segundo fator nessa rodada. Transformamos os Y de volta em 1, para permitir uma nova cópia completa na rodada seguinte. Depois voltamos ao começo e escolhemos a próxima unidade do primeiro fator."

Mostre os resumos abaixo, explicando que são etapas resumidas e não passos consecutivos:

```
X11*11#11      1 rodada:  2 unidades no resultado
XX1*11#1111    2 rodadas: 4 unidades no resultado
XXX*11#111111  3 rodadas: 6 unidades no resultado
```

4.5 Limpar e parar

Estados para apontar: `q_apagar_esquerda`, `q_buscar_sep`, `q_limpar` e `q_fim`.

Fala sugerida:

"Quando não há mais 1 para escolher no primeiro fator, q_escolher encontra o asterisco. Isso inicia a limpeza. Apagamos os X, o asterisco, o segundo fator e o sustenido. O resultado fica preservado e entramos em q_fim. Assim, restam apenas os seis símbolos 1."

Como ler o grafo: um rótulo `1/X,R` significa ler `1`, escrever `X` e mover à direita. O destino da seta indica o próximo estado. Um laço retorna ao mesmo estado, mas ainda executa uma transição. O estado com

contorno duplo é o final.

Frase para justificar a correção: "Cada X corresponde a uma cópia completa do segundo fator no resultado. Com três X, copiamos o segundo fator três vezes."

5. Demonstração ao vivo — 8:30 a 11:30

Execute os comandos um por vez. Aguarde o término e comente o resultado antes de passar ao próximo. Use entradas pequenas para o trace caber no tempo disponível.

5.1 Caso válido e leitura do trace

Comando:

```
python simulador.py "111*11"
```

Fale antes de executar: "Vou executar três vezes dois. O resultado esperado são seis símbolos 1. O console mostra cada configuração da máquina."

Depois de executar, volte ao início da saída para apontar uma configuração:

```
Passo 0 | estado: q0 | cabeçote: 0
B 1 1 1 * 1 1 B
  ^
```

Fale: "Passo zero é a configuração inicial. O estado é q0, a posição do cabeçote é zero, e a seta aponta a célula que está sendo lida. Os B representam os brancos. Cada transição mostra uma nova configuração, inclusive depois dos movimentos."

Volte ao fim da saída e fale: "A execução parou em estado final, deixou 111111 na fita e usou 131 transições. Isso confirma o resultado esperado para esse exemplo."

5.2 Caso base: multiplicação por zero

Comando:

```
python simulador.py "*111"
```

Resultado esperado: `ACEITA/PARADA`, com `Fita final: (vazia)`.

Fala sugerida: "Adotamos a sequência vazia para representar zero em unário. Aqui não há nenhuma unidade antes do asterisco, então estamos calculando zero vezes três. A fita final vazia representa o resultado zero."

5.3 Entrada inválida

Comando:

```
python simulador.py "11**1"
```

Resultado esperado: `REJEITA`.

Fala sugerida: "Esta entrada tem dois separadores. A máquina chega ao estado q_segundo e encontra outro asterisco. Como não existe uma regra para esse par, e o estado não é final, o simulador rejeita a entrada. A validação do formato faz parte das transições."

Se houver tempo, mostre também `python simulador.py "11*2"` e diga: "Neste caso, o símbolo 2 nem pertence ao alfabeto de entrada. O programa informa o erro antes de iniciar a máquina. É diferente da rejeição por formato que acabamos de ver."

6. Limite, testes e fechamento — 11:30 a 13:00

O que executar:

```
python simulador.py "111*11" --limite 10
```

Resultado esperado: `LIMITE DE PASSOS`.

Fala sugerida:

"Agora vou executar a mesma entrada válida com um limite propositalmente pequeno. Como ela precisa de 131 transições, dez não são suficientes. O simulador interrompe a execução. O limite padrão é de cem mil passos e evita que o programa continue indefinidamente quando uma máquina não termina."

"Atingir o limite não prova que existe um loop infinito: neste exemplo sabemos que a máquina terminaria se pudesse continuar. Também não significa rejeitar a entrada."

"Além da demonstração, verificamos automaticamente as 81 combinações de fatores entre zero e oito, casos malformados, o limite e uma máquina auxiliar que escreve dos dois lados da posição inicial. Os quatro testes passaram."

Se pedirem para ver os testes, execute `python testes.py`; o resultado esperado é `OK`.

Fala de encerramento:

"A solução lê as regras do arquivo, simula uma única fita que cresce nos dois sentidos, mostra cada passo e controla a parada. Para a nossa equipe, as regras fazem a multiplicação por cópias repetidas e apagam os símbolos auxiliares, deixando somente o produto."

7. Perguntas individuais — 13:00 a 15:00

Reserve esse tempo para responder. Use as respostas abaixo para estudar antes da apresentação.

Por que a máquina é determinística?

"Porque existe no máximo uma transição para cada par de estado atual e símbolo lido. O carregador recusa regras duplicadas para a mesma chave. Assim, não há escolha entre dois próximos movimentos."

Por que chamamos de transdutor?

"Porque transforma a entrada em uma saída escrita na fita. O objetivo é produzir o produto, além de reconhecer se o formato da entrada é válido."

Qual é a diferença entre X e Y?

"X marca uma unidade do primeiro fator que já iniciou uma rodada. Ele permanece até a limpeza final. Y marca uma unidade do segundo fator que já foi copiada na rodada atual. Ele volta a ser 1 ao terminar essa rodada."

Por que o segundo fator precisa ser restaurado?

"Para que todas as suas unidades possam ser copiadas de novo para cada unidade do primeiro fator. Sem a restauração, as marcas da rodada anterior impediriam novas cópias."

Para que serve o sustenido?

"Para separar o segundo fator da área de resultado. Os dois usam símbolos 1, então precisamos dessa referência para saber onde termina um e começa o outro."

O Python está fazendo a multiplicação diretamente?

"Não. A função simular aplica uma regra por vez, e não calcula o produto. As regras de multiplicação estão no arquivo. Nos testes, o produto calculado em Python é usado somente como referência para conferir a resposta da máquina."

Por que é uma máquina de fita única?

"Porque entrada, marcas e resultado estão no mesmo conjunto de células, acessado por um único cabeçote. Ter áreas com funções diferentes não cria outras fitas."

Como vocês mostram que a fita cresce para a esquerda?

"O cabeçote usa um inteiro que pode ficar negativo. O teste auxiliar escreve X nas posições menos um e um e deixa a posição zero em branco, formando XBX. Isso verifica escrita nos dois lados. Na multiplicação, o cabeçote também visita o branco à esquerda do início para conseguir voltar ao começo."

Por que remover os brancos do dicionário?

"Porque uma chave ausente já é lida como branco. Remover a chave preserva o significado da fita e evita guardar células vazias sem necessidade."

Quando o simulador aceita, rejeita ou interrompe?

"Ele aceita ou para quando chega a um estado listado como final. Rejeita quando não há regra aplicável em um estado não final. Interrompe por limite quando ainda poderia executar uma regra, mas já realizou a quantidade máxima permitida."

O resultado precisa voltar para a posição zero?

"O enunciado pede que o produto fique na fita. Nossa máquina o deixa na área de resultado, com brancos à esquerda. A exibição final omite os brancos externos. Nenhuma marca auxiliar permanece, e o cabeçote termina no início do resultado."

Por que podemos afirmar que o algoritmo termina em entradas válidas?

"Cada rodada marca um novo 1 do primeiro fator. Dentro dela, cada cópia marca um novo 1 do segundo fator. Como os fatores são finitos, essas repetições acabam. Depois há uma limpeza finita. O limite configurado precisa permitir a conclusão."

8. Apoio para revisar o código

Se o professor pedir detalhes, localize a função relacionada ao assunto:

- `carregar_maquina`: lê as linhas, ignora comentários, valida as definições e constrói o dicionário de regras. A chave é `(estado, símbolo lido)`.

- `mostrar_fita`: exibe as células, o estado, o número do passo e a posição do cabeçote. A posição do ^ muda conforme a célula atual.

- `simular`: cria a fita e repete leitura, consulta à regra, escrita, movimento e troca de estado. Também verifica os motivos de parada.

- `main`: recebe entrada, arquivo e limite pelo terminal, carrega a máquina e apresenta o resultado ou uma mensagem de erro.

Frase para memorizar a execução: "Lê o símbolo, procura a regra, escreve, move e troca o estado; depois repete até parar."

9. Resumo de bolso para o ensaio

- Problema: contar unidades; `111*11` significa três vezes dois.

- Fita: dicionário, índices negativos e células ausentes como branco.

- Regras: arquivo `.txt`; uma regra por par de estado e símbolo.

- Estratégia: marcar X, copiar usando Y, restaurar Y, repetir e limpar.

- Símbolos: X controla as rodadas; Y controla as cópias; # separa o resultado.

- Demonstração: produto normal, zero, formato inválido e limite pequeno.

- Parada: final conclui; regra ausente rejeita; limite interrompe.

- Justificativa: cada unidade do primeiro fator gera uma cópia completa do segundo.

Se houver vários apresentadores, distribuam os blocos pelo tempo disponível. Ensaiem as trocas de fala e mantenham todos preparados para explicar cada parte.